12 Coding Standards Part 1: Our Own

Most computer programming is done by organizations and teams. Some programs are developed by lone individuals within a team and others by collaborative groups. In order to promote program quality, many organizations develop formal programming guidelines called *coding standards* that define specific technical and stylistic coding practices to help ensure code quality and consistency.

In this adventure, we're going to develop our own shell script coding standard to be known henceforth as the *LinuxCommand Bash Script Coding Style Guide*. The Source adventure is a suggested prerequisite for this adventure.

Roaming around the Internet, we can find lots of articles about "proper" shell script standards and practices. Some are listed in the "Further Reading" section at the end of this adventure. While the scripts presented in TLCL do not follow any particular standard (instead, they present common practice from different historical perspectives), their design promotes several important ideas:

1. **Write cleanly and simply.** Look for the simplest and most easily understood solutions to problems.
2. **Use modern idioms, but be aware of the old ones.** It's important that scripts fit within common practice so that experienced people can easily understand them.
3. **Be careful and defensive.** Follow Murphy's Law: anything that can go wrong eventually will.
4. **Document your work.**
5. **There are lots of ways to do things, but some ways are better than others.**

Coding standards generally support the same goals, but with more specificity.

In reviewing the Internet's take on shell scripting standards, one might notice a certain undercurrent of hostility towards using the shell as a programming medium at all. Why is this?

Most programmers don't learn the shell at the beginning of their programming careers; instead, they learn it (haphazardly) after they have learned one or more traditional programming languages. Compared to most programming languages, the shell is an odd beast that seems, to many, a chaotic mess. This is largely due to the shell's unique role as both a command line interpreter and a scripting language.

As with all programming languages, the shell has its particular strengths and weaknesses. The shell is good at solving certain kinds of problems, and not so good at others. Like all good artists, we need to understand the bounds of our medium.

What the Shell is Good At

- The shell is a powerful glue for connecting thousands of command line programs together into solutions to a variety of data processing and system administration problems.
- The shell is adept at *batch processing*. In the early days of computing, programs were not interactive; that is, they started, they carried out their tasks, and they ended. This style of programming dominated computing until the early 1960s when disk storage and virtual memory made timesharing and interactive programs possible. Those of us who remember MS-DOS will recall that it had a limp substitute for shell scripts called batch files.

What the Shell is Not So Good At

- The shell does not excel with programs requiring a lot of user interaction. Yes, the shell does have the `read` command and we could use `dialog`, but let's face it, the shell is not very good at this.
- The shell is not suitable for implementing algorithms requiring complex data structures. While the shell does have integers, strings, and one dimensional arrays (which can be associative), it doesn't support anything beyond that. For example, it doesn't have structures, enumerated or Boolean types, or even floating point numbers.

A Coding Standard of Our Own

Keeping the points above in mind, let's make our own shell script coding standard. It will be an amalgam of various published standards, along with a dash of the author's own humble wisdom on the subject. As the name implies, the *LinuxCommand Bash Script Coding Style Guide* coding standard will be very `bash` specific and as such, it will use some features that are not found in strictly POSIX complaint shells.

Script Naming, Location, and Permissions

Like other executables, shell script file names should not include an extension. Shared libraries of shell code which are not standalone executables should have the extension `.sh` if the code is portable across multiple shells (for example `bash` and `zsh`) or `.bash` if the code is bash-specific.

For ease of execution, scripts should be placed in a directory listed in the user's `PATH`. The `~/bin` directory is a good location for personal scripts as most Linux distributions support this out of the box. Scripts intended for use by all users should be located in the `/usr/local/bin` directory. System administration scripts (ones intended for the

superuser) should be placed in `/usr/local/sbin`. Shared code can be located in any subdirectory of the user's home directory. Shared code intended for use system wide should be placed in `/usr/local/lib` unless the shared code specifies only configuration settings, in which case it should be located in `/usr/local/etc`.

Executable code must be both readable and executable to be used, thus the permission setting for shell scripts should be 755, 750 or 700 depending on security requirements. Shared code need only be readable, thus the permissions for shared code should be 644, 640, or 600.

Structure

A shell script is divided into five sections. They are:

1. The shebang
2. The comment block
3. Constants
4. Functions
5. Program body

The Shebang

The first line of a script should be a shebang in either of the following forms:

```
#!/bin/bash
```

Or

```
#!/usr/bin/env bash
```

The second form is used in cases where the script is intended for use on a non-Linux system (such as macOS). The `env` command will search the user's `PATH` for a `bash` executable and launch the first instance it finds. Of the two forms, the first is preferred, as its results are less ambiguous.

The Comment Block

The first bit of documentation to appear in the script is the comment block. This set of comments should include the name of the script and its author, any necessary copyright and licensing information, a brief description of the script's purpose, and its command line options and arguments.

It is also useful to include version information and a revision history. Here is an example of a fully populated comment block. The exact visual style of the block is undefined and is left to the programmer's discretion.

```
# ------------------------------------------------------------------------------
# new_script - Bash shell script template generator
```

```
# Copyright 2012-2021, William Shotts <bshotts@users.sourceforge.net>

# This program is free software: you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.

# This program is distributed in the hope that it will be useful,
# but WITHOUT ANY WARRANTY; without even the implied warranty of
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
# GNU General Public License at <http://www.gnu.org/licenses/> for
# more details.

# Usage: new_script [[-h|--help]
#        new_script [-q|--quiet] [-s|--root] [script]]

# Revision history:
# 2021-04-02  Updated to new coding standard (3.5)
# 2019-05-09  Added support for shell scripting libraries (3.4)
# 2015-09-14  Minor cleanups suggested by Shellcheck (3.3)
# 2014-01-21  Minor formatting corrections (3.2)
# 2014-01-12  Various cleanups (3.1)
# 2012-05-14  Created
# ---------------------------------------------------------------------------
```

Constants

After the comment block, we define the constants used by the script. As we recall, constants are variables that have a fixed value. They are used to define values that are used in various locations in the script. Constants simplify script maintenance by eliminating hard coded values scattered throughout the code. By placing this section at the top, changes to the script can be made more easily.

There are two constants that every script should include. First, a constant that contains the name of the program, for example:

```
PROGNAME=${0##*/}
```

This value is useful for such things as help and error messages. In the example above, the name of the program is calculated from the first word on the command line that invoked the script (`$0`) with any leading path name stripped off. By calculating it this way, if the name of the script file ever changes, the constant will automatically contain the new name.

The second constant that should be included contains the script's version number. Like the `PROGNAME` constant, this value is useful in help and error messages.

```
VERSION="3.5"
```

Constants should also be defined for numerical values such as maximum or minimum limits used in calculations, as well as for the names of files or directories used by, or acted upon, by the script.

Functions

Function definitions should appear next in the script. They should follow an order that indicates their hierarchy; that is, functions should be ordered so they are defined before they are called. For example, if `funct_b` calls `funct_a`, function `funct_a` should be defined ahead of `func_b`.

Functions should have the following form:

```
func_b {

  local v1="$1"
  local v2="$2"

  command1
  command2
  return
}
```

Positional parameters passed to a function should be assigned to local variables with descriptive names to clarify how the parameters are used. Unless there is a good reason, all functions should end with a `return` statement with an exit status as required.

Each function defined in a code library should be preceded by a short comment block that lists the function's name, purpose, and positional parameters.

Program Body

The final section of a script is the program body, where we get to do what we came here to do. In this section we handle such things as positional parameter processing, acquisition and validation of required input, processing, and output.

A common practice is to write the program body as a very simple abstraction of the program, so that the basic program flow is easy to understand and most of the fussy bits are placed in functions defined above the program body.

The end of the program body should include some means of housekeeping, to do such things as removing temporary files. All scripts should return a useful exit status.

Formatting and Visual Style

Good visual layout makes scripts easier to read and understand. It's important to make formatting consistent so it creates a reliable visual language for the code.

Line Length

In order to improve readability and to display properly in small terminal windows, line length should be limited to 80 characters. Line continuation characters should be used to break long lines, with each subsequent line indented one level from the first. For example:

```
printf "A really long string follows here: %s\n" \
  "Some really, really, really long string."
```

Indentation

Indention should be done with spaces and never with tab characters. Most text editors can be set to use spaces for tabbing. In order for the maximum number of characters to be included on each line, each level of indentation should be 2 spaces.

Constant, Variable and Function Names

Constant names should be written in all caps. Variable and function names should be written in all lowercase. When a constant or variable name consists of multiple words, underscore characters should be used as the separator. Camel case (“camelCase”) should be avoided as it makes people think we’re one of those snooty Java programmers just learning to write shell scripts ;-)

Long Command Option Names

When using less common commands with multiple options (or more common commands with less common options), it is sometimes best to use the long form option names and split the option list across multiple lines.

```
# Use long option names to improve readability

rsync \
  --archive \
  --delete-excluded \
  --rsh=ssh \
  --one-file-system \
  --relative \
  --include-from="$INCLUDE_FILE" \
  --exclude-from="$EXCLUDE_FILE" \
  "$SOURCE" "$DESTINATION"
```

Pipelines

Pipelines should be formatted for maximum clarity. If a pipeline will fit on one line cleanly, it should be written that way. Otherwise, pipelines should be broken into multiple lines with one pipeline element per line.

```
# Short pipeline on one line

command1 | command2

# Long pipeline on multiple lines

command1 \
  | command2 \
  | command3 \
  | command4
```

Compound Commands

Here are the recommended formatting styles for compound commands;

```
# 'then' should appear on the same line as 'if'

if [[ -r ~/.bashrc ]]; then
  echo ".bashrc is readable."
else
  echo ".bashrc is not readable." >&2
  exit 1
fi

# Likewise, 'do' should appear on the same line as
# the 'while', 'until', and 'for' commands

while [[ -z "$str" ]]; do
  command1
  command2
done

for i in 1 2 3 4 5; do
  command1
  command2
done

# In a case statement, simple one-line commands can be
# formatted this way:

case s in
  1|one)
    command1 ;;
  2|two)
    command2 ;;
  3|three)
    command3 ;;
  *)
    command4 ;;
esac

# Multiple commands should be formatted this way

case s in
  1|one)
    command1
    command2
    ;;
  2|two)
    command3
    command4
    ;;
  3|three)
    command5
    command6
    ;;
  *)
    command7
    ;;
esac

# Logical compound commands using && and ||
```

```
command1 && short_command
command2 \
  || long_command "$param1" "$param2" "$param3"
```

Coding Practices

In order to achieve our goal of writing robust, easily maintained scripts, our coding standard recommends the following coding practices.

Commenting

Good code commenting is vital for script maintenance. The purpose of commenting to is to explain vital facts about a program. If a script is to have any durability, we must anticipate that someone (even if it's just the author) will revisit the script at a later date and will need a refresher on how the script works. Do not comment code that is obvious and easily understood; rather, explain the difficult to understand parts. Rule of thumb: the more time a chunk of code takes to design and write, the more commenting it will likely need to explain it.

Function libraries are a special case. Each function in a library should be preceded by a comment block that documents its purpose and its positional parameters.

A common type of comment points to where future additions and changes are needed. These are called "todo" comments and are typically written like this:

```
# TODO Fix this routine so it can do this better
```

These comments begin with the string `TODO` so they can be found easily using a text editor's search feature.

Shell Builtins vs. External Programs

Where possible, use bash builtins rather than external commands. For example, the `basename` and `dirname` programs can be replaced by parameter expansions to strip leading or trailing strings from pathnames. Compared to external programs, shell builtins use fewer resources and execute much faster.

Variable Expansion and Quoting

Double quotes must be used to manage word splitting during parameter expansion. This is particularly important when working with filenames. Assume that every variable, parameter expansion, and command substitution may contain embedded spaces, newlines, etc. There are situations where quoting is not needed (for example, within `[[ ... ]]`) but we use double quotes anyway, because it doesn't hurt anything and it's easier to always quote variables than remembering all the special cases where it is not required.

```
a="$var"
```

```
b="$1"
c="$(command1)"
command2 "$file1" "$file2"
[[ -z "$str" ]] || exit 1
```

Contrary to some other coding standards, brace delimiting variables is required only when needed to prevent ambiguity during expansion:

```
a="Compat"
port="bably condit"
echo "${a}bility is pro${port}ional to desire."
```

Pathname Expansion and Wildcards

Since pathnames in Unix-like systems can contain any character except `/` and `NULL`, we need to take special precautions during expansion.

```
# To prevent filesnames beginning with `-` from being interpreted
# as command options, always do this:
command1 ./*.txt

# Not this:
command1 *.txt
```

Here is a snippet of code that will prepend `./` to a pathname when needed.

```
# This will sanitize '$pathname'
[[ "$pathname" =~ ^[./].*$ ]] || pathname="./$pathname"
```

[[…]] vs. […]

Unless a script must run in a POSIX-compatible environment, use `[[ ... ]]` rather than `[ ... ]` when performing conditional tests. Unlike the `[` and `test` bash builtins, `[[ ... ]]` is part of shell syntax, not a command. This means it can handle its internal elements (test conditions) in a more robust fashion, as pathname expansion and word splitting do not occur. Also, `[[ ... ]]` adds some additional capabilities such as `=~` to perform regular expression tests.

Use ((…)) for Integer Arithmetic

Use `(( ... ))` in place of `let` or `exper` when performing integer arithmetic. The bash `let` builtin works in a similar way as `(( ...))` but its arguments often require careful quoting. `exper` is an external program and many times slower than the shell.

```
# Use (( ... )) rather than [[ ... ]] when evaluating integers
if (( i > 1 )); then
  ...
fi

while (( y == 5 )); do
  ...
done

# Perform arithmetic assignment
(( y = x * 2 ))
```

```
# Perform expansion on an arithmetic calculation
echo $(( i * 7 ))
```

printf vs. echo

In some cases, it is preferable to use `printf` over `echo` when parameter expansions are being output. This is particularly true when expanding pathnames. Since pathnames can contain nearly any character, expansions could result in command options, command sequences, etc.

Error Handling

The most important thing for a script to do, besides getting its work done, is making sure it's getting its work done successfully. To do this, we have to handle errors.

1. **Anticipate errors.** When designing a script, it is important to consider possible points of failure. Before the script starts, are all the necessary external programs actually installed on the system? Do the expected files and directories exist and have the required permissions for the script to operate? What happens the first time a script runs versus later invocations? The beginning of the program should include tests to ensure that all necessary resources are available.

2. **Do no harm.** If the script must do anything destructive, for example, deleting files, we must make sure that the script does only the things it is supposed to do. Test for all required conditions prior to doing anything destructive.

3. **Report errors and provide some clues.** When an error is detected, we must report the error and terminate the script if necessary. All error messages must be sent to standard error and should include useful information to aid debugging. A good way to do this is to use an error message function such as the one below:

```
error_exit() {

  local error_message="$1"

  printf "%s\n" "${PROGNAME}: ${error_message:-"Unknown Error"}" >&2
  exit 1
}
```

   We can call the error message function to report an error like this:

```
command1 || error_exit "command1 failed in line $LINENO"
```

   The shell variable `LINENO` is included in the error message passed to the function. This will contain the line number where the error occurred.

4. **Clean up the mess.** When we handle an error we need to make sure that we leave the system in good shape. If the script creates temporary files or performs some

> operation that could leave the system in an undesirable state, provide a way to return the system to useful condition before the script exits.

Bash offers a setting that will to attempt handle errors automatically, which simply means that with this setting enabled, a script will terminate if any command (with some necessary exceptions) returns a non-zero exit status. To invoke this setting, we place the command `set -e` near the beginning of the script. Several bash coding standards insist on using this feature along with a couple of related settings, `set -u` which terminates a script if there is an uninitialized variable, and `set -o PIPEFAIL` which causes script termination if any element in a pipeline fails.

Using these features is not recommended. It is better to design proper error handling and not rely on `set -e` as a substitute for good coding practices.

The *Bash FAQ #105* provides the following opinion on this:

> `set -e` was an attempt to add "automatic error detection" to the shell. Its goal was to cause the shell to abort any time an error occurred, so you don't have to put `|| exit 1` after each important command.
>
> That goal is non-trivial, because many commands intentionally return non-zero. For example,
>
> ```
> if [ -d /foo ]; then ...; else ...; fi
> ```
>
> Clearly we don't want to abort when the `[ -d /foo ]` command returns non-zero (because the directory does not exist) – our script wants to handle that in the else part. So the implementors decided to make a bunch of special rules, like "commands that are part of an if test are immune", or "commands in a pipeline, other than the last one, are immune."
>
> These rules are extremely convoluted, and they still fail to catch even some remarkably simple cases. Even worse, the rules change from one Bash version to another, as bash attempts to track the extremely slippery POSIX definition of this "feature." When a subshell is involved, it gets worse still – the behavior changes depending on whether bash is invoked in POSIX mode.

Command Line Options and Arguments

When possible, scripts should support both short and long option names. For example, a "help" feature should be supported by both the `-h` and `--help` options. Dual options can be implemented with code such as this:

```
# Parse command line
while [[ -n "$1" ]]; do
  case $1 in
    -h | --help)
```

```
      help_message
      graceful_exit
      ;;
    -q | --quiet)
      quiet_mode=yes
      ;;
    -s | --root)
      root_mode=yes
      ;;
    --* | -*)
      usage > &2; error_exit "Unknown option $1"
      ;;
    *)
      tmp_script="$1"
      break
      ;;
  esac
  shift
done
```

Assist the User

Speaking of “help” options, all scripts should include one, even if the script supports no other options or arguments. A help message should include the script name and version number, a brief description of the script’s purpose (as it might appear on a man page), and a usage message that describes the supported options and arguments. A separate usage function can be used for both the help message and as part of an error message when the script is invoked incorrectly. Here are some example usage and help functions:

```
# Usage message - separate lines for mutually exclusive options
# the way many man pages do it.
usage() {
  printf "%s\n" \
    "Usage: ${PROGNAME} [-h|--help ]"
  printf "%s\n" \
    "       ${PROGNAME} [-q|--quiet] [-s|--root] [script]"
}

help_message() {
  cat <<- _EOF_
  ${PROGNAME} ${VERSION}
  Bash shell script template generator.

  $(usage)

  Options:

  -h, --help    Display this help message and exit.
  -q, --quiet   Quiet mode. No prompting. Outputs default script.
  -s, --root    Output script requires root privileges to run.

_EOF_
}
```

Traps

In addition to a normal exit and an error exit, a script can also terminate when it receives a signal. For some scripts, this is an important issue because if they exit in an unexpected manner, they may leave the system in an undesirable state. To avoid this problem, we include traps to intercept signals and perform cleanup procedures before the scripts exits. The three signals of greatest importance are SIGINT (which occurs when Ctrl-c is typed) and SIGTERM (which occurs when the system is shut down or rebooted) and SIGHUP (when a terminal connection is terminated). Below is a set of traps to manage the SIGINT, SIGTERM, and SIGHUP signals.

```
# Trap signals
trap "signal_exit TERM" TERM HUP
trap "signal_exit INT"  INT
```

Due to the syntactic limitations of the `trap` builtin, it is best to use a separate function to act on the trapped signal. Below is a function that handles the signal exit.

```
signal_exit() { # Handle trapped signals

  local signal="$1"

  case "$signal" in
    INT)
      error_exit "Program interrupted by user"
      ;;
    TERM)
      printf "\n%s\n" "$PROGNAME: Program terminated" >&2
      graceful_exit
      ;;
    *)
      error_exit "$PROGNAME: Terminating on unknown signal"
      ;;
  esac
}
```

We use a `case` statement to provide different outcomes depending on the signal received. In this example, we also see a call to a `graceful_exit` function that could provide needed cleanup before the script terminates.

Temporary Files

Wherever possible, temporary files should be avoided. In many cases, process substitution can be used instead. Doing it this way will reduce file clutter, run faster, and in some cases be more secure.

```
# Rather than this:
command1 > "$TEMPFILE"
.
.
.
command2 < "$TEMPFILE"

# Consider this:
command2 < <(command1)
```

If temporary files cannot be avoided, care must be taken to create them safely. We must consider, for example, what happens if there is more than one instance of the script running at the same time. For security reasons, if a temporary file is placed in a world-writable directory (such as `/tmp`) we must ensure the file name is unpredictable. A good way to create temporary file is by using the `mktemp` command as follows:

```
TEMPFILE="$(mktemp /tmp/"$PROGNAME".$$.XXXXXXXXX)"
```

In this example, a temporary file will be created in the `/tmp` directory with the name consisting of the script's name followed by its process ID (PID) and 10 random characters.

For temporary files belonging to a regular user, the `/tmp` directory should be avoided in favor of the user's home directory.

ShellCheck is Your Friend

There is a program available in most distribution repositories called `shellcheck` that performs analysis of shell scripts and will detect many kinds of faults and poor scripting practices. It is well worth using it to check the quality of scripts. To use it with a script that has a shebang, we simply do this:

```
shellcheck my_script
```

ShellCheck will automatically detect which shell dialect to use based on the shebang. For shell script code that does not contain a shebang, such as function libraries, we use ShellCheck this way:

```
shellcheck -s bash my_library
```

Use the `-s` option to specify the desired shell dialect. More information about ShellCheck can be found at its website http://www.shellcheck.net.

Summing Up

We covered a lot of ground in this adventure, specifying a complete set of technical and stylistic features. Using this coding standard, we can now write some serious production-quality scripts. However, the complexity of this standard does impose some cost in terms of development time and effort.

In Part 2, we will examine a program from LinuxCommand.org called `new_script`, a bash script template generator that will greatly facilitate writing scripts that conform to our new coding standard.

Further Reading

Here are some links to shell scripting coding standards. They range from the lax to the obsessive. Reading them all is a good idea in order to get a sense of the community's

collective wisdom. Many are not bash-specific and some emphasize multi-shell portability, not necessarily a useful goal.

- *The Google Shell Style Guide* The coding standard for scripts developed at Google. It's among the most sensible standards. https://google.github.io/styleguide/shellguide.html
- *Anyone Can Write Good Bash (with a little effort)* https://blog.yossarian.net/2020/01/23/Anybody-can-write-good-bash-with-a-little-effort
- *Some Bash coding conventions and good practices* https://github.com/icy/bash-coding-style
- *Bash Style Guide and Coding Standard* https://lug.fh-swf.de/vim/vim-bash/StyleGuideShell.en.pdf
- *Shell Script Standards* https://engineering.vokal.io/Systems/sh.md.html
- *Unix/Linux Shell Script Programming Conventions and Style* http://teaching.idallen.com/cst8177/13w/notes/000_script_style.html
- *Scripting Standards* http://ronaldbradford.com/blog/scripting-standards/

Pages with advice on coding practices. Some have conflicting advice so *caveat emptor*.

- *Make Linux/Unix Script Portable With* `#!/usr/bin/env` *As a Shebang* https://www.cyberciti.biz/tips/finding-bash-perl-python-portably-using-env.html
- *Good practices for writing shell scripts* http://www.yoone.eu/articles/2-good-practices-for-writing-shell-scripts.html
- *Why is* `printf` *better than* `echo`*?* https://unix.stackexchange.com/questions/65803/why-is-printf-better-than-echo
- *Why doesn't* `set -e` *(or* `set -o errexit`*, or* `trap ERR`*) do what I expected?* http://mywiki.wooledge.org/BashFAQ/105
- *Bash: Error handling* https://fvue.nl/wiki/Bash:_Error_handling
- *Writing Robust Bash Shell Scripts* https://www.davidpashley.com/articles/writing-robust-shell-scripts/
- *Filenames and Pathnames in Shell: How to do it Correctly* A good page full of cautionary tales and advice on dealing with "funny" file and path names. A serious problem in Unix-like systems. Highly recommended. https://dwheeler.com/essays/filenames-in-shell.html

- *1963 Timesharing: A Solution to Computer Bottlenecks* This YouTube video from MIT provides some historical perspective on the invention of interactive systems and the transition away from batch processing. The concepts presented here are still the basis of all modern computing. https://youtu.be/Q07PhW5sCEk